ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

CET214

Web Programming

03 - Semantic HTML, Accessibility & CSS Fundamentals



Dr. Ahmed Said

© 2026

Learning Objectives

- Build HTML forms with input types, labels, `<fieldset>`, `autocomplete` and validation attributes
- Write a modern `<head>` : `<meta>` tags and favicons
- Structure pages with HTML5 semantic tags and check them with the W3C Validator
- Apply CSS inline, page-level and from an external stylesheet, and explain why external wins
- Write CSS rules with tag, grouping, universal, descendant, class and ID selectors

Part 1 - HTML Forms

Goal: collect user input with labelled, grouped and validated controls.

HTML Forms

- The `<form>` element creates an interactive section for collecting user input
- Forms are one of the primary ways users interact with websites

```html
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">
  <button type="submit">Submit</button>
</form>
```

Name: Submit

- `action` - the URL where the form data is sent
- `method` - how data is sent (`GET` or `POST`)
- `<label for>` ↔ `<input id>` - connects the text to the control
- `name` - the key sent to the server: submitting "Sara" sends `name=Sara`
- `<button type="submit">` (or `<input type="submit">`) sends the form

GET vs POST

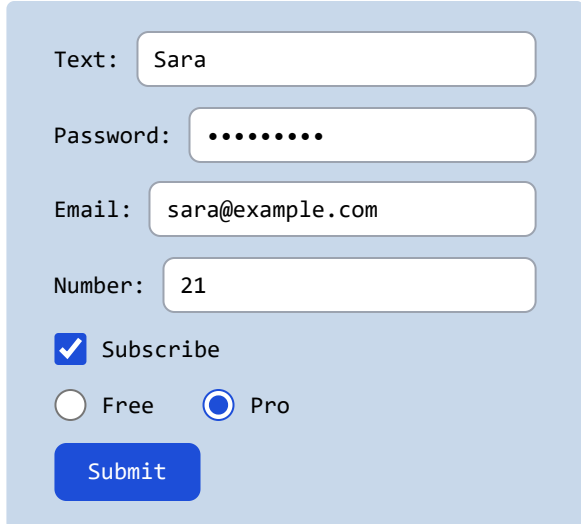
- The `method` attribute specifies how form data is sent to the server
- Two common methods:

Feature	GET	POST
Data location	Appended to URL: `/search?q=html&page=2`	Sent in HTTP request body
Visibility	In address bar, history, server logs	Not in URL (but visible in DevTools)
Data limit	URL length limits (a few KB in practice)	Large (limit set by the server)
Caching & bookmarks	Can be cached & bookmarked	Not cached or bookmarkable
Meaning	Read data - safe to repeat	Change data - resubmit warning
Use case	Search forms, filtering	Login, registration, file uploads

> ⓘ **POST is not encryption**
>
> **Never** use GET for passwords - they end up in the URL, history and logs. But POST alone is **not** secure either: only **HTTPS** encrypts the data.

Common Input Types

```html
<form>
  <label>Text:
    <input type="text" name="nickname" value="Sara">
  </label>
  <label>Password:
    <input type="password" name="pwd" value="secret123">
  </label>
  <label>Email:
    <input type="email" name="email" value="sara@example.com">
  </label>
  <label>Number:
    <input type="number" name="age" value="21" min="0" max="100">
  </label>
  <label>
    <input type="checkbox" name="subscribe" checked> Subscribe
  </label>
  <label><input type="radio" name="plan" value="free"> Free </label>
  <label><input type="radio" name="plan" value="pro" checked> Pro </label>
  <button type="submit">Submit</button>
</form>
```

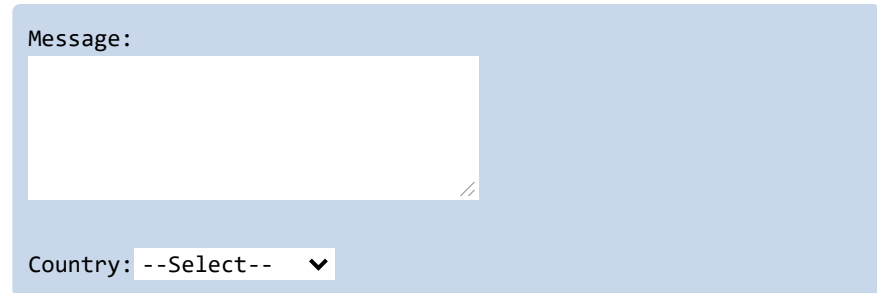


- Radios **share** a `name` so only one can be picked, and each needs its own `value`.
- Other types to try: `date`, `tel`, `url`, `range`, `color`, `file`, `search`.

Labels, Textarea & Select

Labels improve **accessibility** by associating text with form controls.

```html
<form>
  <label for="msg">Message:</label><br>
  <textarea id="msg" name="message"
    rows="4" cols="30"></textarea>
  <br><br>
  <label for="country">Country:</label>
  <select id="country" name="country">
    <option value="">--Select--</option>
    <option value="eg">Egypt</option>
    <option value="sa">Saudi Arabia</option>
    <option value="ae">UAE</option>
  </select>
</form>
```



Tip

Use the `for` attribute on `<label>` matching the input's `id` - clicking the label focuses the input! Keep `</textarea>` on the **same line**: any whitespace inside becomes its default text.

Grouping & Smarter Inputs

```html
<fieldset>
  <legend>Shipping address</legend>
  <label for="street">Street</label>
  <input id="street" name="street"
         autocomplete="street-address">

  <label for="zip">Postal code</label>
  <input id="zip" name="zip"
         inputmode="numeric"
         autocomplete="postal-code">
</fieldset>
```

- `<fieldset>` + `<legend>` - group related controls; screen readers announce the legend ("Shipping address, Street…"). Essential for radio groups!
- `autocomplete` - lets the browser fill known data: `name`, `email`, `tel`, `street-address`, `postal-code`, `current-password`, `new-password`
- `inputmode` - picks the on-screen keyboard (`numeric`, `decimal`, `tel`, `email`) without changing validation
- Use `type="number"` only for real quantities - postal codes, card numbers and phone numbers are **text**

Form Validation Attributes

- HTML5 provides built-in form validation - no JavaScript needed!
  - `required` - the field must be filled in before submission
  - `pattern` - a regex the **whole** value must match
  - `min` / `max` - minimum and maximum values for number inputs
  - `minlength` / `maxlength` - character length constraints
  - `type="email"` / `type="url"` - validate the format automatically
- `placeholder` is only a hint, **not** a label - it disappears when the user types

```html
<form>
  <label for="user">Username (3–20 letters or digits)</label>
  <input id="user" name="username" required minlength="3" maxlength="20" pattern="[A-Za-z0-9]+" autocomplete="username">
  <label for="mail">Email</label>
  <input id="mail" type="email" name="email" required placeholder="you@example.com">
  <label for="age">Age</label>
  <input id="age" type="number" name="age" min="18" max="120">
  <button type="submit">Register</button>
</form>
```

> ⓘ **Validate on the server too**

Try It Yourself: Registration Form

Build a form (8 minutes) - then try to submit it **empty** and with **bad data**:

1. A `<fieldset>` with `<legend>Account</legend>`: username (3–20 chars) and email - both `required`
2. A `<fieldset>` with `<legend>Plan</legend>`: two radio buttons, **Free** and **Pro**
3. A phone field: `type="tel"`, `autocomplete="tel"`
4. A `<textarea>` "About you", max 200 characters
5. Every control has a visible `<label>`; submit with a `<button>`

✅ Self-check: click each label - does it focus its field? Press `Tab` through the form - does the order make sense? Does the browser block an empty username?

Part 2 - Web Page Metadata

Goal: write the `<head>` that browsers, search engines and social apps actually use.

Web Page Metadata: `<meta>`

- Meta tags provide **information about your page** for browsers, search engines, and social media
- Placed in the `<head>` section (they do not appear on the page itself)
  - **Charset**: Text encoding - must come first in `<head>`
  - **Viewport**: Controls layout on mobile devices
  - **Description**: A brief summary often shown in search engine results
  - **Robots**: Instructions for crawlers - only needed to block (`noindex` , `nofollow`)

```html
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Semantic HTML Guide - CET214</title>
  <meta name="description" content="Learn HTML5 semantic tags and accessibility">
</head>
```

SEO Tip

- A unique `<title>` and `<meta name="description">` matter most.
- `<meta name="keywords">` is **ignored** by Google (since 2009) - skip it.



Part 3 - HTML5 Semantic Tags

Goal: replace "div soup" with elements that describe what each region of the page is.

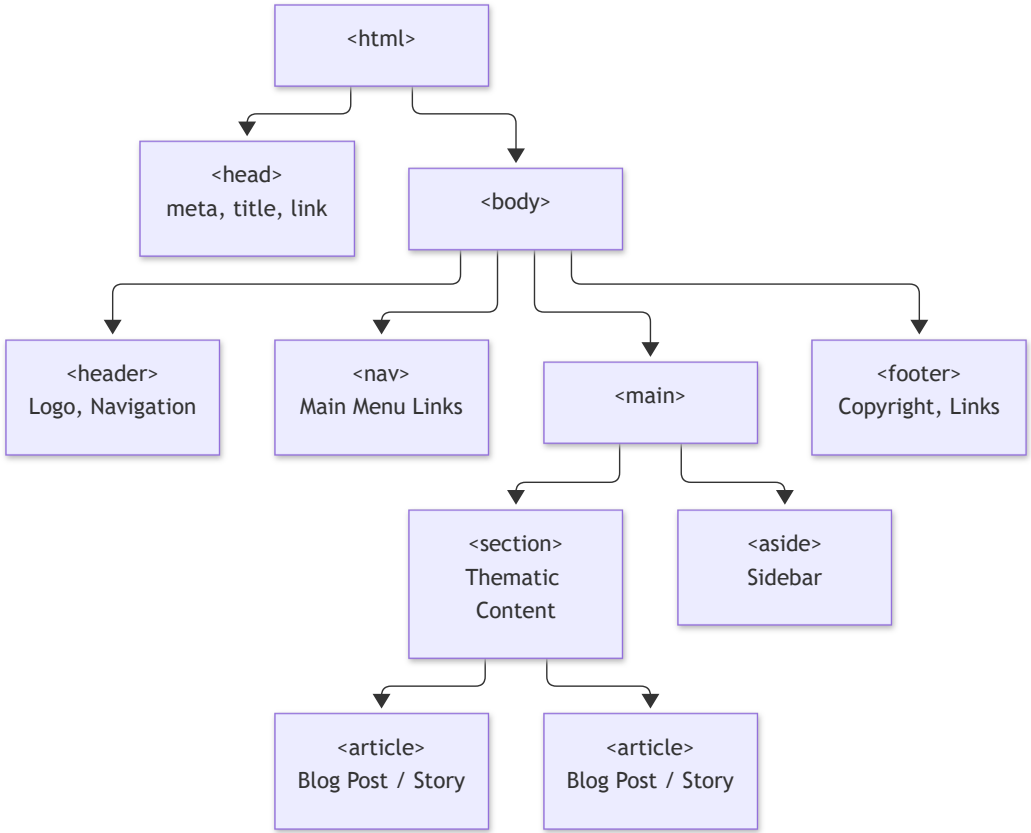
What Are Semantic Tags?

- **Semantic HTML** means using tags that describe the **meaning** and **purpose** of content, not just its appearance.
- Tags like `<header>`, `<footer>`, `<main>`, `<nav>`, `<section>`, `<article>`, and `<aside>` convey structure
- Benefits:
  - **Accessibility**: Screen readers understand page structure
  - **SEO**: Search engines better interpret content
  - **Maintainability**: Code is clearer and easier to read
  - **Interoperability**: Works consistently across browsers and devices

Fun Fact

Before HTML5, developers used `<div id="header">`, `<div id="nav">`, etc. Semantic tags replaced this pattern with meaningful elements!

HTML5 Structure Diagram



General Outline with HTML5

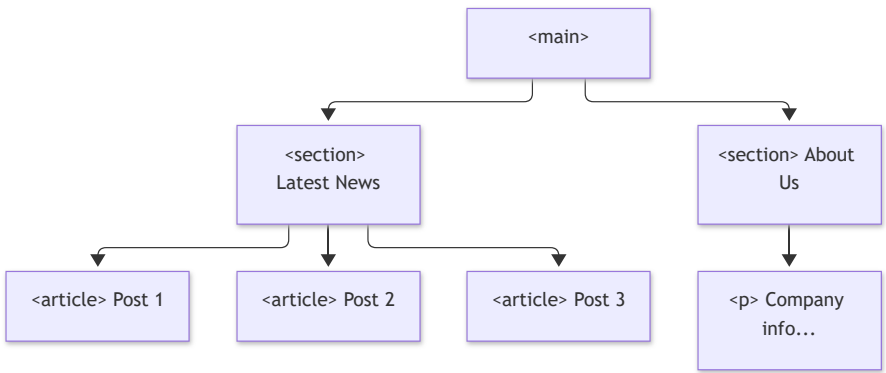
```html
<body>
  <header>
    <!-- Header: logo, site title, navigation bar -->
  </header>
  <nav>
    <!-- Primary navigation links -->
  </nav>
  <main>
    <section>
      <!-- A thematic grouping of content -->
      <article>
        <!-- Self-contained, independently distributable content -->
      </article>
    </section>
    <aside>
      <!-- Sidebar: related links, ads, supplementary info -->
    </aside>
  </main>
  <footer>
    <!-- Footer: copyright, contact info, secondary links -->
  </footer>
</body>
```

Semantic Tags Reference

- `<header>` - introductory content or navigation aids for the page or a section
- `<footer>` - footer content: copyright, contact info, links
- `<main>` - the dominant content of the `<body>` (only **one** visible per page)
- `<nav>` - a section of navigation links (can have multiple per page)
- `<section>` - a thematic grouping of content, typically with a heading
- `<article>` - self-contained content that can be independently distributed (blog post, news story, comment)
- `<aside>` - content tangentially related to the surrounding content (sidebars, pull quotes)
- Also useful: `<figure>` / `<figcaption>` (image + caption), `<time datetime>`, `<address>`, `<search>`

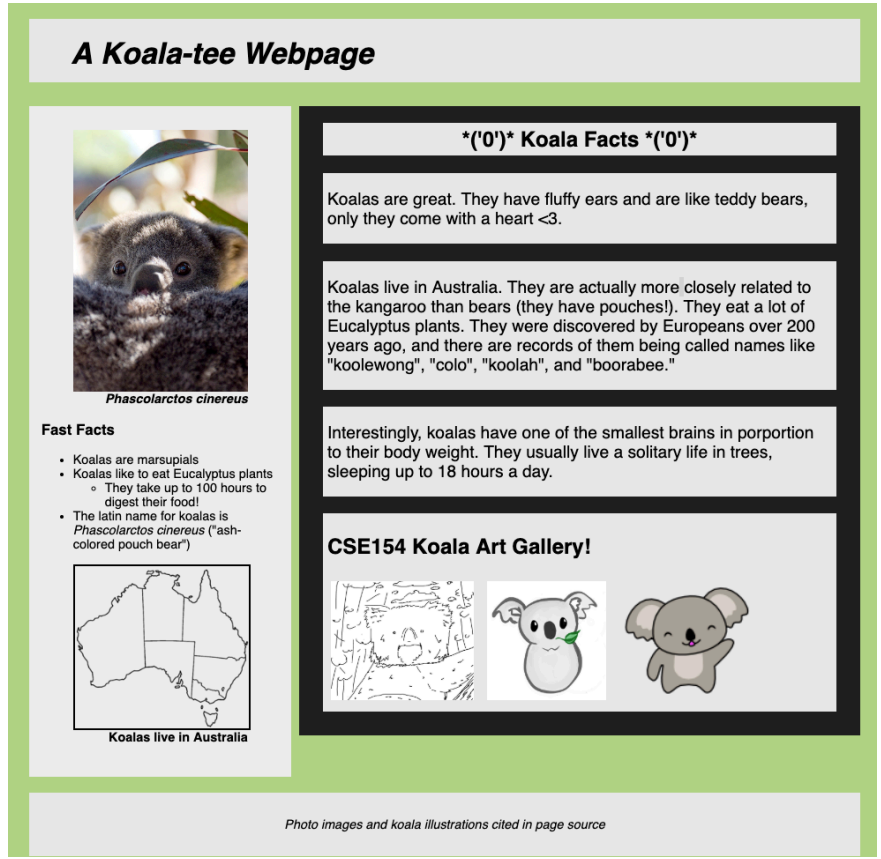
<article> vs <section>

Feature	<article>	<section>
Purpose	Self-contained, independent content	Thematic grouping of related content
Reusability	Can stand alone (e.g., in an RSS feed)	Depends on surrounding context
Examples	Blog post, news story, forum post, comment	Chapter, tab panel, grouped form fields
Heading	Should have a heading	Should have a heading
Nesting	Can contain <section> elements	Can contain <article> elements



Semantic Layout Example

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Koala Fan Page</title>
  </head>
  <body>
    <header>
      <h1>A Koala-tee Webpage</h1>
    </header>
    <main>
      <aside>
        <!-- Left sidebar -->
      </aside>
      <section>
        <!-- Koala facts -->
        <article>
          <!-- Koala art gallery -->
        </article>
      </section>
    </main>
    <footer>
      <!-- Image citations -->
    </footer>
  </body>
</html>
```



Try It Yourself: Fix the Div Soup

Rewrite this page using semantic elements (5 minutes):

```html
<div id="top">
  <div class="logo">Cairo Eats</div>
  <div class="menu">
    <a href="/">Home</a> <a href="/menu">Menu</a>
  </div>
</div>
<div id="content">
  <div class="post">
    <div class="title">Best Koshari in Town</div>
    <div>Posted <span>1 March 2026</span></div>
  </div>
</div>
<div id="bottom">© 2026 Cairo Eats</div>
```

```html
<header>
  <p>Cairo Eats</p>
  <nav>
    <a href="/">Home</a> <a href="/menu">Menu</a>
  </nav>
</header>
<main>
  <article>
    <h2>Best Koshari in Town</h2>
    <p>Posted <time datetime="2026-03-01">1 March 2026</time></p
  </article>
</main>
<footer>© 2026 Cairo Eats</footer>
```

<> Nesting Tags Correctly

Tags must be properly nested - a closing tag must match the **most recently opened** tag.

```html
<!-- ✅ Correct -->
<p>This is <em>really, <strong>REALLY</strong></em> fun!</p>

<!-- ❌ Incorrect -->
<p>This is <em>really, <strong>REALLY</em> lots of</strong> fun!</p>
```

ⓘ Invalid but rendered

The browser **may still render** incorrectly nested HTML by guessing a fix - different browsers may guess differently, and screen readers get a broken structure. It is **invalid**: always validate your code!

Quick Quiz: Nesting Tags

Can you find **every error** in this code?

```
1  <nav>
2      <ul>
3        <li>Home</li>
4        <li>About
5          <ul>
6            <li>Team</li>
7            <li>Contact</li>
8          </ul>
9        </li>
10     </ul>
11 </nav>
12 <article>
13     <h2>Instructions</h2>
14     <p>Can you find every error in this code?</p>
15     <div>
16       <p>Unclosed paragraph</p>
17     </div>
18     <section>
19       <p>
20         <em>Important <strong>note</strong> for students</em>
21       </p>
22     </section>
23 </article>
```

(line numbers of the original code)

- Line 4: `<li>About` never closed - its `</li>` belongs after the inner `</ul>`
- Line 7: `<li>Contact` missing `</li>`
- Line 10: `<nav>` should be `</nav>` (missing slash)
- Line 15: `<p>` is not closed
- Line 19: `<em>` and `<strong>` overlap - close `</strong>` before `</em>`

`<div>` and `<span>`

- `<div>` is a generic **block** container. `<span>` is a generic **inline** container.
- Neither has any meaning or default look - they exist as **styling hooks**.

```html
<div style="background: #e0f0ff;
            padding: 10px;">
  <h3>A div container</h3>
  <p>
    This is a paragraph with a
    <span style="color: red;
      font-weight: bold;">
      highlighted span
    </span> inside it.
  </p>
</div>
```

A div container

This is a paragraph with a **highlighted span** inside it.

Best Practice

- Use semantic tags (`<section>` , `<article>` , `<nav>`) instead of `<div>` when the content has a clear meaning.
- Use `<div>` and `<span>` only as styling/layout hooks - with a CSS `class` , not inline `style` (used here only to show the boxes; Part 4 explains why).

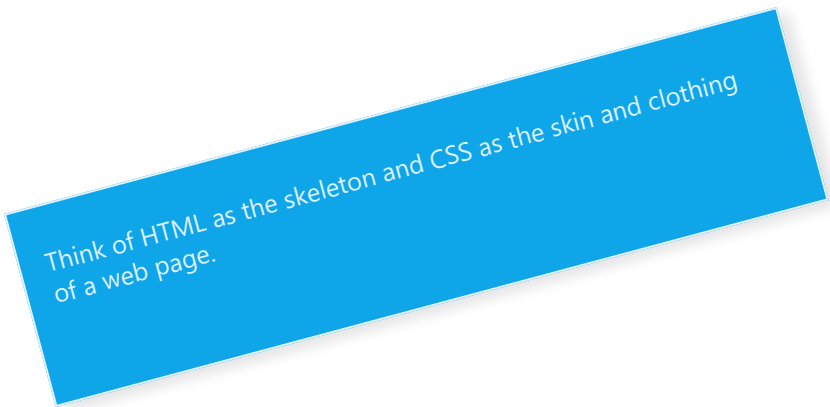


Part 4 - CSS Fundamentals

Goal: separate presentation from content and write CSS rules with selectors, classes and IDs.

What is CSS?

- **Cascading Style Sheets (CSS)** control how the browser renders HTML elements
- CSS describes the **appearance and layout** of information on a web page
- HTML describes **content** - CSS describes **presentation**
- Using CSS, you can tell the browser to render any element differently than its default
- CSS provides tremendous flexibility in controlling the look of your pages



❗ The Bad Way to Produce Styles (1990s HTML)

Before CSS, authors styled pages with presentational tags. `<font>` is obsolete - removed from HTML5; never use it today:

```html
<p>
  <font face="Arial">Welcome to Greasy Joe's.</font>
  You will <b>never</b>, <i>ever</i>, <u>EVER</u> beat
  <font size="+4" color="red">OUR</font> prices!
</p>
```

Welcome to Greasy Joe's. You will **never**, ever, EVER beat OUR prices!

- **Accessibility** - screen readers cannot interpret visual-only styling
- **Code organization** - mixes content with presentation
- **Maintainability** - changing style requires editing every single tag
- **Today:** `<b>`, `<i>`, `<u>` still exist but carry meaning, not looks - for importance/emphasis prefer `<strong>` / `<em>`, and do all visual styling in CSS

🎨 Three Ways to Apply CSS

Goal: attach CSS to a page with the `style` attribute, a `<style>` block, or an external stylesheet - and know which to prefer.

1 Inline Styles - The `style` Attribute

The `style` attribute adds CSS directly to an HTML element.

```html
<h2 style="font-family: Verdana, sans-serif;">Heading</h2>
<h2>Same Heading without style</h2>
```



- Style declarations follow the pattern: `property: value;`
- Multiple declarations separated by semicolons
- The trailing semicolon is optional but recommended

> ⓘ **Note**
>
> - Inline styles are the **least efficient** method.
> - If you want to bold every paragraph, you would need `style="font-weight: bold"` on every `<p>` tag - no better than using `<b>` tags! They also **beat almost every stylesheet rule**, which makes them hard to override later (Cascade & Specificity lecture).

2 Page-Level Styles - The `<style>` Tag

Place a `<style>` block inside the `<head>` element to apply styles page-wide.

```html
<head>
  <style>
    h1 { font-size: x-large; font-weight: bold; }
    h2 { font-size: large; font-weight: bold; }
  </style>
</head>
```

- The `<style>` tag contains CSS rules that define styles for the entire page
- Place it inside the `<head>` element
- No `type="text/css"` needed - CSS is the default in HTML5 (you will still see it in old code)
- Better than inline styles - one rule can affect **all** matching elements
- Limitation: the rules only apply to **this one page**

3 External Stylesheets - The `<link>` Tag

The most efficient way to apply styles to a page or an entire site.

```html
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

- An external stylesheet is a **separate** `.css` **file** containing CSS rules
- Linked to HTML using the `<link>` tag in the `<head>`
- `href` - path to the CSS file
- `rel="stylesheet"` - defines the relationship
- Change one file, update the **entire site's** appearance
- Browsers **cache** external stylesheets for faster loading

Best Practice

External stylesheets are the industry standard for production websites.

Try It Yourself: Three Ways, One Result

Make **every** `<h2>` on a page dark blue and bold - three different ways:

1. **Inline:** add `style="color: navy; font-weight: bold;"` to each `<h2>` - count how many edits you needed
2. **Page-level:** remove those attributes and write **one** rule in a `<style>` block in the `<head>`
3. **External:** move that rule into `style.css` and link it with `<link rel="stylesheet" href="style.css">`

✅ The rule is the same every time: `h2 { color: navy; font-weight: bold; }` - only where it lives changes. Now add a second HTML page and link the same file: both pages update together.

</> CSS Syntax & Selectors

Goal: read and write a CSS rule, and target elements by tag, group, context, and universal selector.

</> Basic CSS Syntax

A CSS rule consists of a **selector** and a **declaration block**.

```css
selector {
  property: value;
  property: value;
}
```

```css
p {
  color: red;
  font-family: sans-serif;
}
```

- The selector points to the HTML element(s) you want to style
- The declaration block is wrapped in curly braces `{}`
- Each declaration has a **property** name and a **value**, separated by a colon
- Declarations are separated by semicolons
- Comments look like `/* ... */` - there is no `//` comment in CSS

Tag Selectors

The simplest type of selector - selects **all** elements of a particular type.

```css
strong {
  color: red;
}
```

This makes **all** `<strong>` text red across the page.

Grouping selectors - apply the same styles to several element types with a comma-separated list:

```css
p, ol, ul { color: blue; }
```

```css
/* ...is shorthand for: */
p  { color: blue; }
ol { color: blue; }
ul { color: blue; }
```

✳ Universal & Body Selectors

- The **universal selector** `*` selects every element on the page
- The **body selector** applies page-wide settings to all visible content

```css
* {
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  color: #333333;
}
```

> **Note**
>
> The universal selector `*` is commonly used in CSS resets to remove default browser margins and padding for a consistent starting point. The modern reset also adds `box-sizing: border-box` - you will see why in lecture 04.

Contextual (Descendant) Selectors

Apply styles to elements **only** when nested within other elements.

```css
ol em { color: blue; }
```

- No comma between selectors means **descendant relationship**
- This rule applies only to `<em>` elements **inside** `<ol>` elements
- Compare with a comma (grouping): `ol, em { color: blue; }` - applies to **both** independently

```css
p cite  { font-style: italic; font-weight: normal; }
li cite { font-style: normal; font-weight: bold; }
```

- `<cite>` inside `<p>` will be italic
- `<cite>` inside `<li>` will be bold
- Semantic structure pays off: `nav a`, `footer p`, `article h2` need no classes

CSS Specificity - A First Look

- When two rules set the **same property** on the **same element**, the more specific selector wins
- This is CSS specificity

```css
/* Rule 1: applies to ALL cite elements */
cite {
  color: green;
}

/* Rule 2: more specific -- cite inside p */
p cite {
  font-style: italic;
  color: red;
}

/* Rule 3: more specific -- cite inside li */
li cite {
  font-weight: bold;
  color: blue;
}
```

- `<cite>` tags not in `<p>` or `<li>` remain **green**
- `<cite>` inside `<p>` becomes **red** - `p cite` names 2 elements, `cite` only 1
- `<cite>` inside `<li>` becomes **blue** (Rule 3 overrides Rule 1 the same way)

ID Classes & IDs

Goal: create your own reusable style hooks with classes, and know when (rarely) an ID is the right tool.

Classes - `.classname`

- When tag selectors are not specific enough, create your own classifications.

```html
<p class="warning">This is a warning.</p>
<p>This is a normal paragraph.</p>
```

```css
.warning {
  color: red;
  font-weight: bold;
}
```

This is a warning.

This is a normal paragraph.

- Use a leading `.` (dot) before the class name in your CSS - but **no dot** in the HTML attribute
- Multiple elements can share the same class
- One element can have **several** classes: `class="warning large"`
- Classes work with contextual selectors too: `ul li.important { color: red; }`

IDs - `#idname`

- An ID (identifier) names a single, unique element on a page.

```html
<p id="important">This is important.</p>
```

```css
#important {
  color: red;
  font-weight: bold;
}
```

This is important.

- Use a leading `#` (hash) before the ID name in your CSS
- An ID must be unique on the page - only one element can have that ID
- IDs are also used heavily in **JavaScript** (`getElementById`), for `<label for="...">` , and for `#fragment` links
- Modern practice: **style with classes**; ID selectors are very specific and hard to override (Cascade & Specificity lecture)

Mixing Selectors

You can combine different selector types in a single rule:

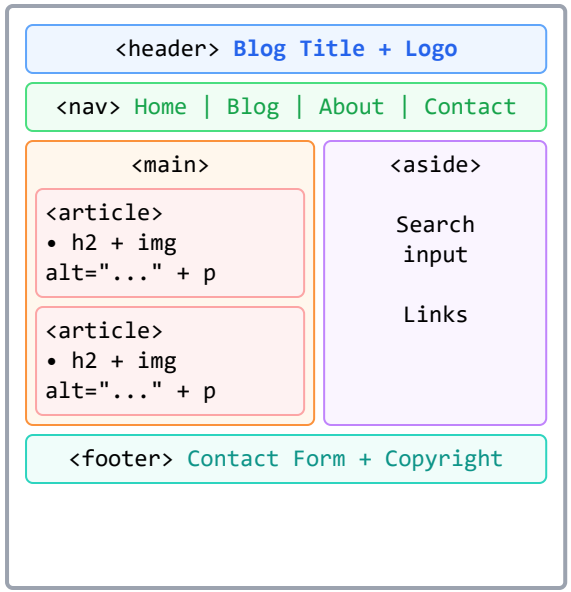
```css
h1, #headline, .heading, .warning {
  font-size: large;
  color: green;
}
```

- Tag selectors (`h1`)
- ID selectors (`#headline`)
- Class selectors (`.heading` , `.warning`)
- All combined in one comma-separated rule

> **Note**
> This is perfectly valid and very useful when you want to set the same properties across multiple different selectors.

Practical Assignment (1/2): Semantic Blog Page

Build a **semantic blog page** (`blog.html`) following this layout:



- Use the **diagram** to plan your layout and ensure proper nesting of semantic tags
- **Head**: `charset` , `viewport` , unique `<title>` and `description`
- **Navigation**: `<nav>` with a list of 4+ links
- **Articles**: Each with heading, paragraph, image with descriptive `alt` ; decorative images get `alt=""`
- **Sidebar**: a labelled search `<input type="search">`
- **Contact form**: `<label>` on every field, `<fieldset>` / `<legend>` , `autocomplete` , validation attributes
- **Check**: 0 errors in the W3C Validator

Practical Assignment (2/2): Style It with `style.css`

Style the same blog page - **no inline styles**, all rules in one external stylesheet:

1. Link `style.css` with `<link rel="stylesheet">`; start with a `*` reset and page defaults on `body`
2. **Tag** and **grouping** selectors, plus at least one **descendant** selector (e.g. `nav a`, `article h2`)
3. At least **two classes**, one element with **both**; an ID only as a hook (e.g. an in-page link target)

Fonts and the box model are added to this same `style.css` in lecture 04, accessibility in lecture 04b.

> **Note**
> Validate your CSS at jigsaw.w3.org/css-validator and your HTML at validator.w3.org before submitting.

THANK YOU